



FOR JAVA DEVELOPERS MOVING TO NESTJS

Deployed. Payment killed.

Spring Boot to NestJS, episode 38: health checks and graceful shutdown

Graceful shutdown has three separate jobs: stop advertising readiness, finish the work in flight, then exit.

A SIGTERM half a second into a three second payment. Nest as created exited in milliseconds; the payment got a connection reset.

Spring Boot 4.1 with Actuator and nothing configured finished the payment and reported readiness 503 while it drained.

enableShutdownHooks finished the payment on Nest but reset new requests at once. A readiness check plus Terminus's delay gave Nest the same shape as Spring.

In scope

- Actuator health and probes
- Terminus
- In-flight requests on SIGTERM
- Readiness during the drain

Not in scope

- Signals and PID 1 in containers (episode 35)
- Kubernetes probe tuning

At the time of recording Spring Boot 4.1 defaults `server.shutdown` to `graceful` and enables the health probes.

01 The measurements

SPRING 4.1, DEFAULTS	payment 200; readiness 503 while serving
NEST AS CREATED	payment reset; gone in milliseconds
NEST, enableShutdownHooks()	payment 200; new requests reset
NEST, + READINESS + TERMINUS DELAY	payment 200; readiness 503 while serving

SIGTERM 500 ms into a 3 s payment.

02 All three jobs on Nest

Enable shutdown hooks, fail readiness from beforeApplicationShutdown, and give the router time with Terminus's graceful shutdown delay.

```
main.ts
```

```
app.enableShutdownHooks();
```

```
readiness.ts
```

```
@Injectable()
export class Readiness implements BeforeApplicationShutdown {
  shuttingDown = false;
  beforeApplicationShutdown() {
    this.shuttingDown = true; // the readiness check now reports 503
  }
}
```

```
app.module.ts
```

```
TerminusModule.forRoot({ gracefulShutdownTimeoutMs: 2000 })
```

payment 200; readiness 503 while new requests were still served; then refused

Run on a real machine, 2026-10-01, by `scripts/verify-health.sh` in the course repository.

Before you trust a deploy

1. Does readiness fail as soon as shutdown begins?
2. Does the app keep serving long enough for the router to notice?
3. Do in-flight requests finish?
4. Does the signal reach the process in the container?
5. Is the shutdown timeout shorter than the orchestrator's grace period?



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- Every measurement, one command: scripts/verify-health.sh**
<https://github.com/code-with-sam-dev/spring-to-node>
- NestJS health checks (Terminus)**
<https://docs.nestjs.com/recipes/terminus>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.